Instituto Tecnológico de Mérida, Yucatán

Carrera: Ingeniería en Sistemas Computacionales

Asignatura: Ingeniería de Software

Examen

Unidad #3

6SA

Profesor: ISC. Rodrigo Leal Ojeda

Alumnos:

Amaris Pricila Celis Magaña

Aldebarán Antonino Galaviz Medina

Fares de Jesús Zuleta Barrios

Juan Pablo Uicab Moo

Valeria Naranjo Cohuo

Sherlin Guadalupe Can Poot

Naomi Isabel May Canul

Entrega: 10/Noviembre/2026

Documento de desarrollo y construcción: Pomodoro

Índice de contenido

1. Selección e implementación del lenguaje de programación (Subtema 3.1) 3
   1.1 Justificación técnica con base en SWEBOK 3
   1.2 Aplicación rigurosa del paradigma (POO y funcional) 4
   1.3 Requisitos funcionales y no funcionales completados en la Unidad 3 6
2. Manejador de bases de datos (DBMS) y persistencia (Subtema 3.2) 7
   2.1 Integración del DBMS local (SQLite v3) y del servidor 7
   2.2 Migraciones y esquema físico implementado (DDL y DML) 9
3. Estándares de codificación y calidad de construcción (Subtemas 3.3.1 y 3.3.5) 10
4. Pruebas unitarias, verificación de caja blanca y mocking (Subtema 3.3.2) 11
   4.1 Estrategia de verificación y aislamiento por simulación (Mocking) 11
   4.2 Catálogo de pruebas ejecutadas y resultados 12
5. Control de versiones, integración continua (CI) y gestión de configuración (Subtemas 3.3.3 y 3.3.4) 14
   5.1 Gestión de configuración bajo ISO/IEC/IEEE 12207 y flujo de ramas 14
   5.2 Pipeline de Integración Continua automatizada (GitHub Actions) 16

Referencias 17

1. Selección e implementación del lenguaje de programación (Subtema 3.1)

1.1 Justificación técnica con base en SWEBOK

Conforme al área de conocimiento Software Construction del SWEBOK (IEEE, 2014), la construcción del sistema Pomodoro emplea Dart 3 con el SDK de Flutter para la aplicación cliente multiplataforma y PHP 8.3/8.4 con el framework Laravel 13 para la API REST de sincronización. La siguiente tabla fundamenta la elección frente a los criterios de dominio, rendimiento, sistema de tipos y ecosistema de bibliotecas.

Criterio SWEBOK	**Cliente: Dart 3 / Flutter**	**Servidor: PHP 8.3–8.4 / Laravel 13**
Adecuación al dominio	Compilación nativa multiplataforma (Android 8+, iOS 13+ y escritorio) desde una sola base de código (RNF-09), con acceso directo a notificaciones programadas del sistema operativo y almacenamiento local SQLite (RNF-08).	Construcción declarativa de servicios web RESTful en JSON sin estado (stateless), con middleware de limitación de intentos (throttle:10,1), autenticación por tokens personales y transacciones ACID (RF-08, RF-09).
Rendimiento	Compilación AOT (Ahead-of-Time) a código máquina nativo que garantiza un tiempo de respuesta de interfaz <= 200 ms en el percentil 95 (RNF-01) y cálculo exacto del temporizador contra el reloj real (RNF-03).	Motor Zend de PHP 8.3/8.4 con OPcache y procesamiento por lotes dentro de una única transacción de base de datos para sostener una latencia <= 500 ms (p95) con 100 usuarios concurrentes (RNF-02).
Sistema de tipos	Tipado estático fuerte con sound null safety (distinción estricta entre tipos anulables como String? o DateTime? y no anulables), eliminando excepciones de referencia nula en tiempo de ejecución.	Firmas con tipos explícitos en parámetros, propiedades y valores de retorno (JsonResponse, Response, array, int, ?PersonalAccessToken), complementadas con casts automáticos en los modelos Eloquent.

Criterio SWEBOK	Cliente: Dart 3 / Flutter	Servidor: PHP 8.3–8.4 / Laravel 13
Ecosistema de bibliotecas	Paquetes oficiales y verificados: sqflite y sqflite_common_ffi (SQLite), flutter_local_notifications (avisos en segundo plano), shared_preferences, audioplayers, uuid, http y flutter_test.	Eloquent ORM (consultas parametrizadas), sistema de migraciones versionadas, fachada Hash (bcrypt de 12 rondas), RateLimiter, PHPUnit 12 y Laravel Pint (PSR-12).

1.2 Aplicación rigurosa del paradigma (POO y funcional)

El código explota las capacidades multiparadigma de Dart y PHP combinando Programación Orientada a Objetos (encapsulamiento, polimorfismo, abstracción e inyección de dependencias) con programación funcional (inmutabilidad, funciones puras y transformaciones declarativas de colecciones), logrando alta modularidad y una separación limpia de responsabilidades entre presentación (lib/screens), dominio (lib/services) y datos (lib/data):

Principio aplicado	Módulo / Archivo	Implementación en el proyecto
Encapsulamiento y ocultamiento de estado	lib/services/ciclo_pomodoro.dart (CicloPomodoro)	Las marcas de tiempo internas (_inicioReloj, _segundosAlIniciar, _inicioSesion, _tareaSesion) y los tiempos pendientes (_pendienteTrabajo, _pendienteDescanso, _pendienteDescansoLargo, _pendienteCiclos) son privados. La interfaz gráfica solo invoca operaciones de dominio (iniciar, sincronizar, pausar, cancelar, finalizarFase, aplicarTiempos).

Principio aplicado	Módulo / Archivo	Implementación en el proyecto
Inmutabilidad y objetos de valor	SesionCompletada, BarraGrafica, ResumenEstadisticas, ResultadoSync	Clases inmutables definidas con constructores const y atributos final. Garantizan que los resultados de dominio no puedan alterarse accidentalmente desde la capa de presentación.
Funciones puras y estilo funcional	lib/services/estadisticas_service.dart (EstadisticasService)	Métodos estáticos puros (agrupar, total, resumir, minutosPorDiaDe, desglosePorAnioMesDia) sin efectos secundarios. Reciben la fecha actual como parámetro opcional para pruebas deterministas y usan operadores funcionales (fold, where, map, collection-for).
Polimorfismo, inyección de dependencias y modularidad	lib/services/sync_service.dart (SyncService)	El constructor de SyncService recibe opcionalmente el cliente HTTP (http.Client) y los delegados de persistencia (exportarDatos y limpiarEliminadas). Aplica polimorfismo de subtipos (http.Client sustituible por IOClient o MockClient), sobrescritura polimórfica (@override) en WidgetsBindingObserver y relación polimórfica tokenable en PersonalAccessToken, logrando separación limpia de responsabilidades.

Principio aplicado	Módulo / Archivo	Implementación en el proyecto
Patrón Singleton con soporte de pruebas	lib/data/database_helper.dart (DatabaseHelper)	Garantiza una única conexión abierta a pomodoro.db en ejecución normal y expone el método @visibleForTesting usarBaseEnMemoria() para ejecutar pruebas de caja blanca sobre SQLite en memoria.

1.3 Requisitos funcionales y no funcionales completados en la Unidad 3

Todos los elementos que los documentos de Análisis (Unidad 1, v1.1) y Diseño (Unidad 2, v1.1) especificaron como diseñados para construirse en la Unidad 3 fueron implementados y verificados en el código:

Requisito	Especificación en Análisis y Diseño v1.1	Implementación realizada en la Unidad 3
RF-02, RF-03, E6	Descanso largo (15-30 min), ciclos antes del descanso largo (2-8, por defecto 4), rango de descanso corto de 3-15 min y aplicación de cambios en la siguiente fase (CA-03.2).	Implementado en CicloPomodoro (minutosDescansoLargo, ciclosAntesDeLargo, ciclosCompletados, esDescansoLargo) y en SettingsScreen. Si hay una fase en curso, aplicarTiempos() difiere el cambio hasta finalizarFase().
RF-04, CA-02.1, CA-02.2	Tareas con prioridad (Alta=1, Media=2, Baja=3), pomodoros estimados editables (1-20) y validación de título de 1 a 120 caracteres.	Implementado en TasksScreen con selector de prioridad, control numérico de estimación (- / +), límite de 120 caracteres y aviso explícito «El título no puede quedar vacío».

Requisito	Especificación en Análisis y Diseño v1.1	Implementación realizada en la Unidad 3
RF-08, RNF-06	Registro, inicio y cierre de sesión con tokens personales de Sanctum (expiración de 24 h), hash bcrypt y límite de 10 intentos por minuto.	Implementado en AuthController (register, login, logout), modelo PersonalAccessToken, migración personal_access_tokens, guard sanctum y pantalla SyncScreen.
RF-09, RNF-07, RNF-10	Sincronización de tareas, sesiones y tareas_eliminadas por lotes con resolución last-write-wins, verificación de propiedad (422) y transacción.	Implementado en SyncController::syncData (con verificarTareasPropias y lockForUpdate), SyncService y DatabaseHelper (exportarParaSync y limpiarEliminadas).
RNF-05	Modo oscuro con Material 3 y contraste accesible en Estadísticas.	Implementado en main.dart (theme y darkTheme con ColorScheme.fromSeed) y en StatsScreen usando colores semánticos del tema en lugar de colores fijos.
RNF-11, CU-02	Confirmación antes de descartar un pomodoro de trabajo en curso y atajos de teclado en escritorio.	Implementado en TimerScreen (_confirmarYCancelar con el diálogo «¿Descartar este pomodoro? No se guardará en el historial.») y widgets Shortcuts/Actions (Espacio y Esc).

2. Manejador de bases de datos (DBMS) y persistencia (Subtema 3.2)

2.1 Integración del DBMS local (SQLite v3) y del servidor

El sistema integra SQLite en dos niveles complementarios bajo principios ACID: en el cliente móvil/escritorio (pomodoro.db, versión 3) como fuente primaria offline-first (ADR-001), y en el servidor Laravel (database.sqlite, ADR-009) con migraciones Eloquent portables a PostgreSQL.

Aspecto técnico	SQLite local en el cliente (pomodoro.db v3)	DBMS del servidor Laravel (SQLite / PostgreSQL)
Volumen y almacenamiento	Tablas tareas y sesiones creadas con WITHOUT ROWID para evitar duplicar el índice primario sobre llaves UUID TEXT, y PRAGMA auto_vacuum = INCREMENTAL para mantener el archivo bajo el límite de 50 MB.	Almacenamiento normalizado en 3FN / FNBC (users, tasks, pomodoro_sessions, personal_access_tokens). Las sesiones no duplican user_id; el dueño se obtiene a través de la tarea.
Concurrencia y rendimiento	PRAGMA journal_mode = WAL (Write-Ahead Logging) para lecturas no bloqueantes e índice idx_sesiones_tarea que acelera el LEFT JOIN de estadísticas.	Índices en tasks(updated_at), tasks(user_id), pomodoro_sessions(updated_at) y personal_access_tokens(expires_at). Bloqueo pesimista por fila con lockForUpdate() durante la sincronización.
Transacciones y consistencia	Uso de db.transaction en la migración onUpgrade (v2 a v3) y en eliminarTarea (borra la tarea y registra el tombstone en tareas_eliminadas de forma atómica).	DB::transaction envuelve toda la petición POST /api/sync: si cualquier sesión apunta a una tarea ajena, lanza ValidationException (422) y revierte todos los cambios previos.
Seguridad activa, consultas parametrizadas y gestión de usuarios/privilegios	Todas las operaciones DML (insert, update, delete, query) usan marcadores posicionales (?) con whereArgs, además de restricciones CHECK en el DDL físico.	Validación estricta de tipos, rangos y UUID en el controlador, consultas parametrizadas de Eloquent ORM contra inyección SQL (RNF-07) y gestión estricta de usuarios y privilegios (mínimo privilegio: tokens Sanctum 24 h, aislamiento por user_id, verificación de propiedad en verificarTareasPropias con rechazo 422 y credenciales en .env).

2.2 Migraciones y esquema físico implementado (DDL y DML)

En el cliente, DatabaseHelper (lib/data/database_helper.dart) actualiza el esquema a la versión 3 aplicando restricciones CHECK de dominio y la tabla tareas_eliminadas sin perder los datos existentes en instalaciones previas (mediante onUpgrade transaccional):

```
CREATE TABLE tareas_eliminadas (
   id TEXT PRIMARY KEY NOT NULL,
   eliminada_en TEXT NOT NULL
);

CREATE TABLE tareas (
   id TEXT PRIMARY KEY NOT NULL,
   titulo TEXT NOT NULL CHECK (length(titulo) BETWEEN 1 AND 120),
   completada INTEGER NOT NULL DEFAULT 0 CHECK (completada IN (0, 1)),
   prioridad INTEGER NOT NULL DEFAULT 2 CHECK (prioridad BETWEEN 1 AND 3),
   pomodoros_estimados INTEGER NOT NULL DEFAULT 1 CHECK (pomodoros_estimados BETWEEN 1 AND 20),
   updated_at TEXT NOT NULL
) WITHOUT ROWID;
```

En el servidor (backend/database/migrations/), el esquema físico se construye mediante seis migraciones versionadas y ordenadas cronológicamente:

Archivo de migración	**Responsabilidad en el esquema físico**
0001_01_01_000000_create_users_table.php	Crea la tabla users (id, name, email UNIQUE, password con hash bcrypt, timestamps).
2026_10_17_000001_create_tasks_table.php	Crea la tabla tasks con llave primaria UUID, llave foránea user_id con ON DELETE CASCADE y marca updated_at(6) indexada.

Archivo de migración	Responsabilidad en el esquema físico
2026_10_17_000002_create_pomodoro_sessions_table.php	Crea la tabla pomodoro_sessions con llave primaria UUID, llave foránea tarea_id con ON DELETE CASCADE y updated_at(6) indexado.
2026_10_21_000001_quitar_user_id_de_pomodoro_sessions.php	Elimina la columna redundante user_id de pomodoro_sessions para garantizar la Tercera Forma Normal (3FN / FNBC).
2026_10_22_000001_agregar_prioridad_a_tasks.php	Agrega la columna prioridad (unsignedSmallInteger, por defecto 2) y el índice tasks_user_id_index.
2026_10_22_000002_create_personal_access_tokens_table.php	Crea la tabla polimórfica personal_access_tokens con hash SHA-256 único y fecha de expiración expires_at (24 h).

3. Estándares de codificación y calidad de construcción (Subtemas 3.3.1 y 3.3.5)

El proyecto aplica las guías de estilo oficiales de cada lenguaje: Effective Dart en el cliente Flutter y PSR-12 en el servidor Laravel, con nombres descriptivos, comentarios pertinentes sin redundancia (Pressman), baja complejidad ciclomática, IDEs avanzados (VS Code / Android Studio), gestores de dependencias (pub / Composer) y análisis estático sin errores.

Estándar / Herramienta	Archivo de configuración	Reglas aplicadas y resultado de verificación
Effective Dart + flutter_lints 6.0	analysis_options.yaml	Activa package:flutter_lints/flutter.yaml junto con prefer_single_quotes, prefer_final_locals, avoid_empty_else y cancel_subscriptions. Nombres en camelCase para miembros, PascalCase para clases y snake_case para archivos. Resultado de flutter analyze: No issues found!
PSR-12 + Laravel Pint 1.27	backend/pint.json	Configurado con preset psr12. Verifica sangría de 4 espacios, llaves de clases y métodos, declaraciones de tipos e importaciones ordenadas. Resultado de ./vendor/bin/pint --test: PASS.
Baja complejidad ciclomática, comentarios pertinentes (Pressman), IDEs y gestores de dependencias	lib/services/* y app/Http/Controllers/*	Se eliminó el código comentado innecesario y se extrajo la construcción de jerarquías de fechas de StatsScreen hacia EstadisticasService.desglosePorAnioMesDia, manteniendo métodos breves de baja complejidad ciclomática, comentarios pertinentes evitando redundancias (Pressman), apoyo de IDEs avanzados (VS Code, Android Studio), gestores de dependencias (pubspec.yaml, composer.json) y build tools (Flutter SDK, Gradle).

4. Pruebas unitarias, verificación de caja blanca y mocking (Subtema 3.3.2)

4.1 Estrategia de verificación y aislamiento por simulación (Mocking)

La suite de verificación consta de 30 pruebas automatizadas (23 en Flutter con flutter_test y 7 en Laravel con PHPUnit 12) diseñadas bajo enfoque de caja blanca y TDD, logrando una cobertura de código representativa (--coverage) que ejercita toda la lógica de negocio crítica y los casos límite (0 sesiones, límites de 1-120 caracteres, restricciones CHECK, bloqueo 429 y tarea ajena 422). Para aislar las dependencias

externas (red HTTP y base de datos), SyncService se prueba inyectando MockClient del paquete http/testing.dart y funciones simuladas de exportación y limpieza.

```
// Fragmento de test/sync_service_test.dart: aislamiento de API y BD con MockClient
final mockHttp = MockClient((request) async {
  expect(request.url.path, '/api/sync');
  expect(request.headers['Authorization'], 'Bearer token-activo');
  return http.Response(jsonEncode({'creados': 2, 'actualizados': 1, 'omitidos': 0}), 200);
});

final service = SyncService(
  client: mockHttp,
  exportarDatos: () async => {'tasks': [...], 'sessions': [], 'tareas_eliminadas': ['id-1']},
  limpiarEliminadas: (ids) async => eliminadasLimpiadas = ids,
);
```

4.2 Catálogo de pruebas ejecutadas y resultados

Archivo de prueba	Casos	Caminos lógicos de caja blanca verificados
test/ciclo_pomodoro_test.dart	8 pruebas	Bloqueo de inicio sin tarea; descuento exacto contra el reloj real; conservación de tiempo al pausar; generación de SesionCompletada al cerrar trabajo; transición descanso -> trabajo; cancelación sin guardar sesión; aplicación diferida de tiempos cuando el reloj corre (CA-03.2); activación de descanso largo tras completar los ciclos configurados y reinicio del contador (E6).

Archivo de prueba	Casos	Caminos lógicos de caja blanca verificados
test/sync_service_test.dart	4 pruebas (con Mocks)	Inicio de sesión exitoso guardando token en SharedPreferences (MockClient 200); manejo de bloqueo por límite de 10 intentos por minuto (MockClient 429); envío de payload con limpieza de tareas_eliminadas tras respuesta 200; preservación de borrados pendientes cuando la API responde 422 por tarea ajena.
test/database_helper_test.dart	3 pruebas (SQLite RAM)	Inserción y exportación de tareas con prioridad y pomodoros estimados; borrado transaccional de tarea con eliminación en cascada de sus sesiones y registro de tombstone en tareas_eliminadas; rechazo de datos fuera de rango por restricciones CHECK del DDL.
test/estadisticas_service_test.dart	4 pruebas	Agrupación de minutos en 14 barras diarias, 8 semanales (desde lunes) y 12 mensuales en UTC, además del caso límite sin sesiones.
test/resumen_estadisticas_test.dart	4 pruebas	Cálculo de minutos totales, sesiones de hoy, racha de días consecutivos en UTC, agrupación de tareas eliminadas y desglose de minutos por día de una tarea.

Archivo de prueba	Casos	Caminos lógicos de caja blanca verificados
backend/tests/Feature/AuthTest.php	3 pruebas (PHPUnit)	Registro de usuario en POST /api/register con hash bcrypt y emisión de token en personal_access_tokens (201); rechazo de credenciales inválidas en POST /api/login (401); revocación efectiva del token en POST /api/logout (204).
backend/tests/Feature/SyncTest.php	2 pruebas + 2 base	Sincronización de tareas y sesiones en POST /api/sync verificando conteos y persistencia en 3FN (200); rechazo con código 422 y reversión transaccional cuando un usuario intenta registrar una sesión asociada a la tarea de otro usuario (escenario E4).

5. Control de versiones, integración continua (CI) y gestión de configuración (Subtemas 3.3.3 y 3.3.4)

5.1 Gestión de configuración bajo ISO/IEC/IEEE 12207 y flujo de ramas

El control de versiones se administra con Git en el repositorio oficial (https://github.com/GalavizMedinaAA04/PomodoroApp) aplicando flujo GitFlow (main v1.0.0, develop y 7 ramas feat/INT1 a feat/INT7 con contribuciones equitativas de los 7 integrantes) y mensajes de commit atómicos y semánticos bajo la convención Conventional Commits, garantizando trazabilidad completa entre cada cambio de código, su modelo de diseño y sus pruebas:

Commit / Rama	Tipo semántico	Elementos de configuración integrados
feat/INT1-proyecto-base (GalavizMedinaAA04 · 357a20e..c11e22c)	chore / feat / docs	Configuración base de Flutter (pubspec.yaml, analysis_options.yaml), plataformas nativas, punto de entrada lib/main.dart (modo claro/oscuro Material 3) y README.md.

Commit / Rama	Tipo semántico	Elementos de configuración integrados
feat/INT2-data-layer (EyesLight6 · ac3ac7b, c712af0)	feat(data)	Inicialización SQLite multiplataforma (sqlite_init_io.dart, sqlite_init_web.dart) y DatabaseHelper v3 con WITHOUT ROWID, WAL, CHECK, tareas_eliminadas y transacciones.
feat/INT3-services-layer (riacouoh · 62026d6..d9cacbf)	feat(services)	Servicios de dominio: CicloPomodoro (descanso largo, ciclos y cambios diferidos), NotificationService, EstadisticasService y SyncService con inyección de dependencias.
feat/INT4-timer-screen (sherlinwiii · 22a896b, 9387ace)	feat(assets / ui)	Recurso sonoro assets/sounds/beep.wav y pantalla principal TimerScreen con temporizador contra reloj real, confirmación de descarte y atajos de teclado (Espacio y Esc).
feat/INT5-tasks-stats-screens (Nao-Rust · 095a7fc..7bd1e73)	feat(ui)	Pantallas secundarias: TasksScreen (prioridad y estimación), StatsScreen, GraficasScreen, SyncScreen y SettingsScreen (descanso largo y ciclos).
feat/INT6-api-backend (AmarisCelis · a5e3f9d..caa624c)	feat(api)	API REST en Laravel: configuración con Sanctum 24h y Pint (PSR-12), migraciones en 3FN, modelos Eloquent, AuthController, SyncController y rutas /api.
feat/INT7-tests-ci-docs (fareszuleta · 73768d2..9ac39c6)	ci / test / docs	Pipeline CI (.github/workflows/ci.yml), suites de pruebas unitarias (test/ con MockClient y backend/tests/ con PHPUnit), ADR-001 a ADR-010 y documentación SWEBOK.

5.2 Pipeline de Integración Continua automatizada (GitHub Actions)

El archivo .github/workflows/ci.yml ejecuta automáticamente en cada push y pull_request dos trabajos independientes en contenedores Ubuntu: flutter-ci (análisis estático con flutter analyze y ejecución de las 23 pruebas unitarias con cobertura) y laravel-ci (verificación de estándar PSR-12 con Laravel Pint, migraciones sobre SQLite y ejecución de las pruebas de API con PHPUnit).

```
name: CI - Pomodoro App & API
on:
  push:
    branches: [main, develop, 'feat/**']
  pull_request:
    branches: [main, develop]
jobs:
  flutter-ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: subosito/flutter-action@v2
      - run: flutter pub get
      - run: flutter analyze
      - run: flutter test --coverage
  laravel-ci:
    runs-on: ubuntu-latest
    defaults: { run: { working-directory: backend } }
    steps:
      - uses: actions/checkout@v4
      - uses: shivammathur/setup-php@v2
      - run: composer install --no-interaction --prefer-dist
      - run: ./vendor/bin/pint --test
      - run: php artisan test
```

Referencias

Bourque, P., & Fairley, R. E. (Eds.). (2014). Guide to the Software Engineering Body of Knowledge (SWEBOK Version 3.0). IEEE Computer Society.

Flutter Team. (2026). Effective Dart: Style, Usage, and Design Guidelines. Google. https://dart.dev/effective-dart

International Organization for Standardization [ISO], International Electrotechnical Commission [IEC], & Institute of Electrical and Electronics Engineers [IEEE]. (2017). Systems and software engineering — Software life cycle processes (ISO/IEC/IEEE 12207:2017).

Kendall, K. E., & Kendall, J. E. (2011). Análisis y diseño de sistemas (8.ª ed.). Pearson Educación.

Laravel. (2026). Laravel 13.x Documentation: Testing, Database Transactions & Pint. https://laravel.com/docs

PHP Framework Interop Group [PHP-FIG]. (2019). PSR-12: Extended Coding Style. https://www.php-fig.org/psr/psr-12/

Pressman, R. S. (2010). Ingeniería del software: Un enfoque práctico (7.ª ed.). McGraw-Hill.

Sommerville, I. (2016). Ingeniería de software (10.ª ed.). Pearson Educación.